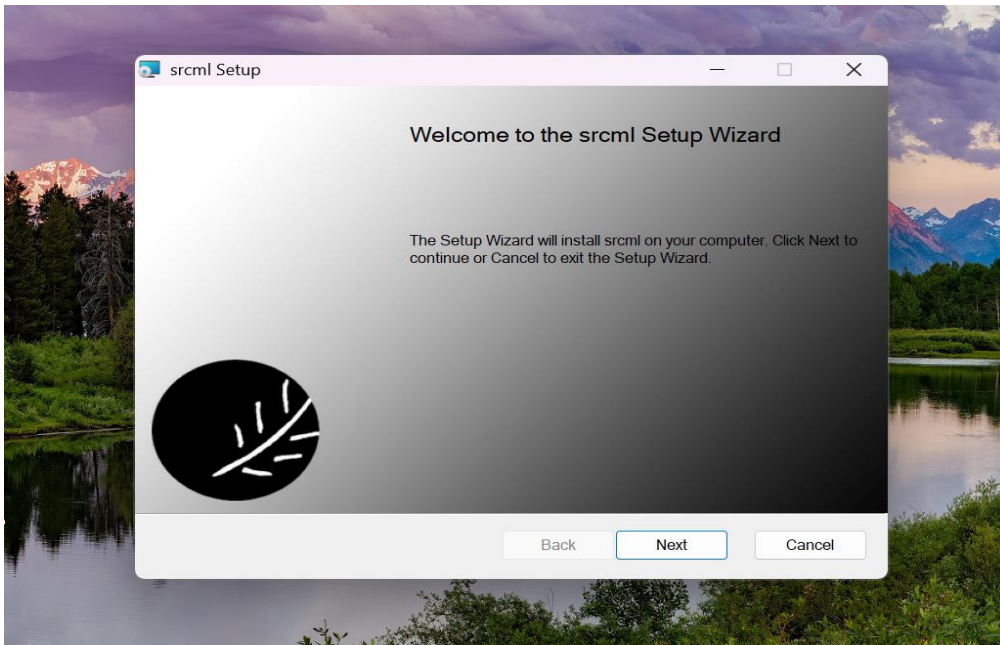
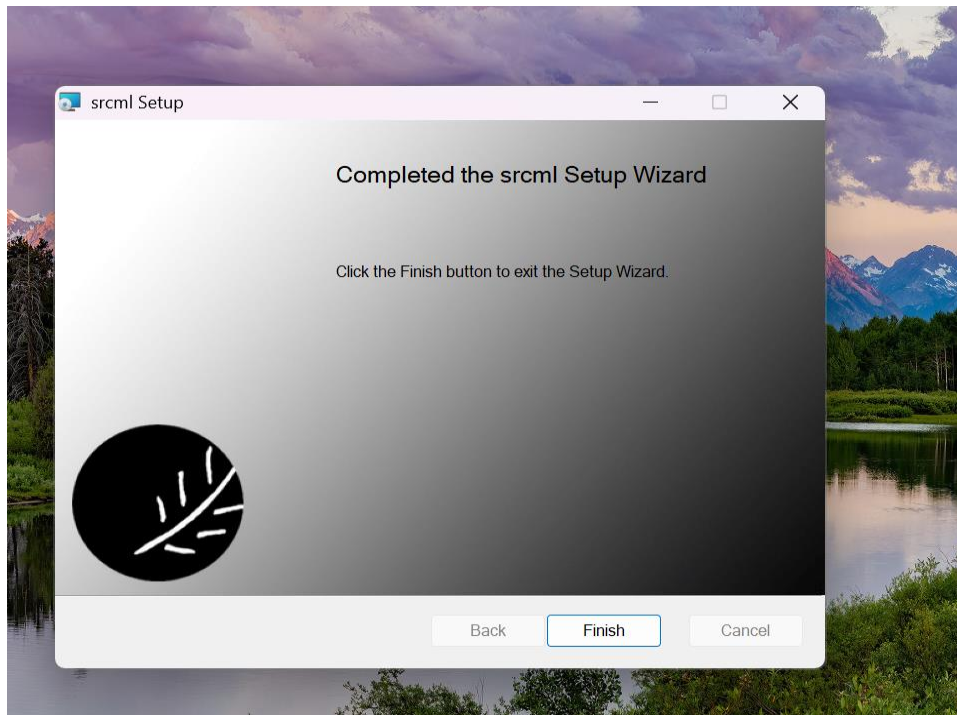


IS-698 Homework 3 – UB01976

Task: Static Analysis and Gen AI

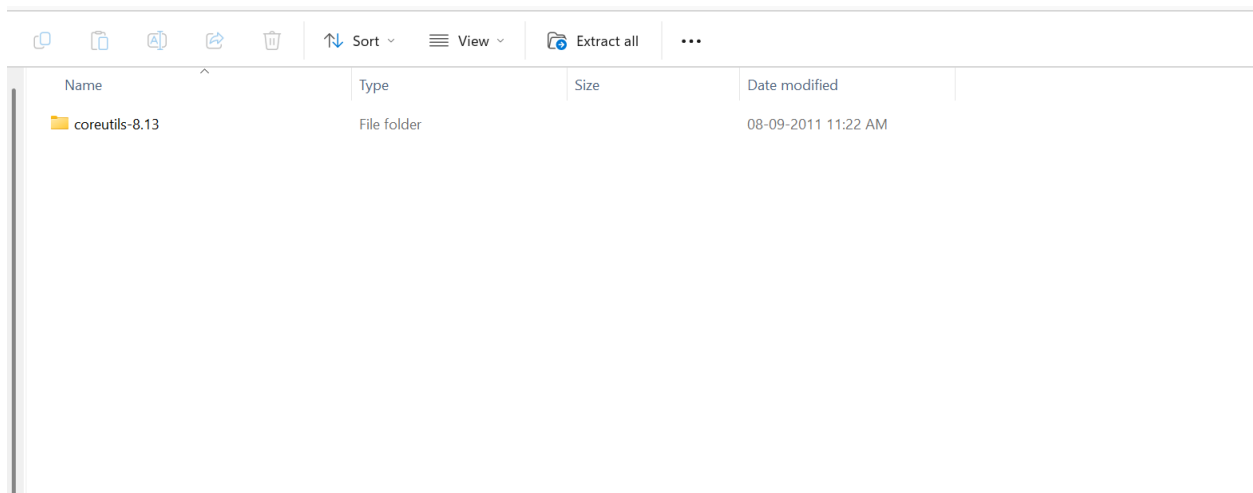
Step 1: The SRCML (Source Markup Language) tool is used to convert or format the source code into the XML document. The version 1.1 is downloaded and installed in the system, please find below the screenshots of the SRCML installing procedure;





Step 2: Downloading coreutils code

For this homework assignment I have downloaded the coreutils 8.13 version



Step 3: Using SRCML to get the total number of functions and function names

First, I have converted the source code file from coreutils into XML file using SRCML. Please find below the screenshots of the conversion process and the XML file;

(The coreutils file downloaded is the zip folder, so before running the conversion all the files from the folder are extracted)

```
Command Prompt
C:\Users\puppa>tar -xvf "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils-8.13.tar.gz" -C "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2"
x coreutils-8.13/
x coreutils-8.13/src/
x coreutils-8.13/src/echo.c
x coreutils-8.13/src/join.c
x coreutils-8.13/src/base64.c
x coreutils-8.13/src/c99-to-c89.diff
x coreutils-8.13/src/extent-scan.h
x coreutils-8.13/src/chown-core.h
x coreutils-8.13/src/mkfifo.c
x coreutils-8.13/src/yes.c
x coreutils-8.13/src/uname-uname.c
x coreutils-8.13/src/nl.c
x coreutils-8.13/src/setuidgid.c
x coreutils-8.13/src/shuf.c
x coreutils-8.13/src/remove.h
x coreutils-8.13/src/cat.c
x coreutils-8.13/src/pr.c
x coreutils-8.13/src/dcggen
x coreutils-8.13/src/stty.c
x coreutils-8.13/src/df.c
x coreutils-8.13/src/seq.c
x coreutils-8.13/src/chroot.c
x coreutils-8.13/src/Makefile.in
x coreutils-8.13/src/timeout.c
x coreutils-8.13/src/uname-arch.c
x coreutils-8.13/src/lbracket.c
x coreutils-8.13/src/ls-ls.c
x coreutils-8.13/src/shred.c
x coreutils-8.13/src/od.c
x coreutils-8.13/src/chown.c
```

Then the conversion of the SRC file to XML file;

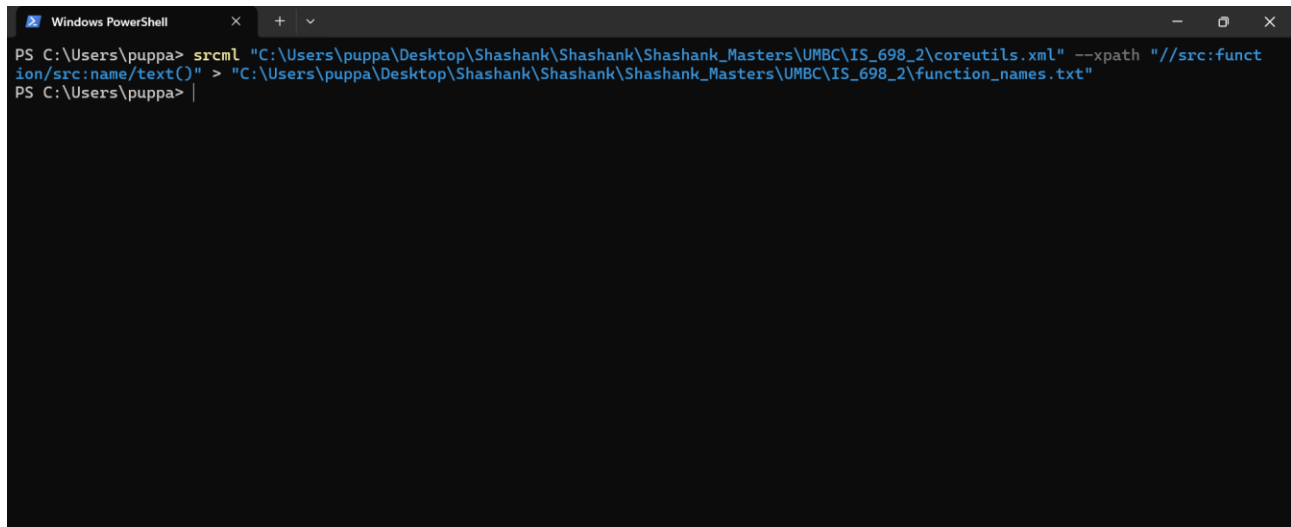
```
Command Prompt
28-07-2011 06:38 AM 67,337 tail.c
28-07-2011 06:38 AM 5,795 tee.c
28-07-2011 06:38 AM 22,371 test.c
01-09-2011 01:16 PM 14,147 timeout.c
28-07-2011 06:38 AM 13,315 touch.c
28-07-2011 06:38 AM 60,426 tr.c
28-07-2011 06:38 AM 2,250 true.c
28-07-2011 06:38 AM 11,613 truncate.c
28-07-2011 06:38 AM 14,153 tsort.c
28-07-2011 06:38 AM 3,138 tty.c
03-07-2011 03:09 PM 48 uname-arch.c
03-07-2011 03:09 PM 49 uname-uname.c
28-07-2011 06:38 AM 10,025 uname.c
04-06-2011 05:21 AM 146 uname.h
28-07-2011 06:38 AM 15,395 unexpand.c
28-07-2011 06:38 AM 16,165 uniq.c
28-07-2011 06:38 AM 2,482 unlink.c
28-07-2011 06:38 AM 6,795 uptime.c
28-07-2011 06:38 AM 3,694 users.c
28-07-2011 06:38 AM 23,877 wc.c
28-07-2011 06:38 AM 2,648 wheel-gen.pl
08-09-2011 10:28 AM 21 wheel-size.h
08-09-2011 10:28 AM 1,878 wheel.h
28-07-2011 06:38 AM 22,434 who.c
28-07-2011 06:38 AM 2,523 whoami.c
28-07-2011 06:38 AM 2,407 yes.c
139 File(s) 2,329,066 bytes
2 Dir(s) 226,063,183,872 bytes free

C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils-8.13>srcml . --position -o "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils.xml"

C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils-8.13>
```

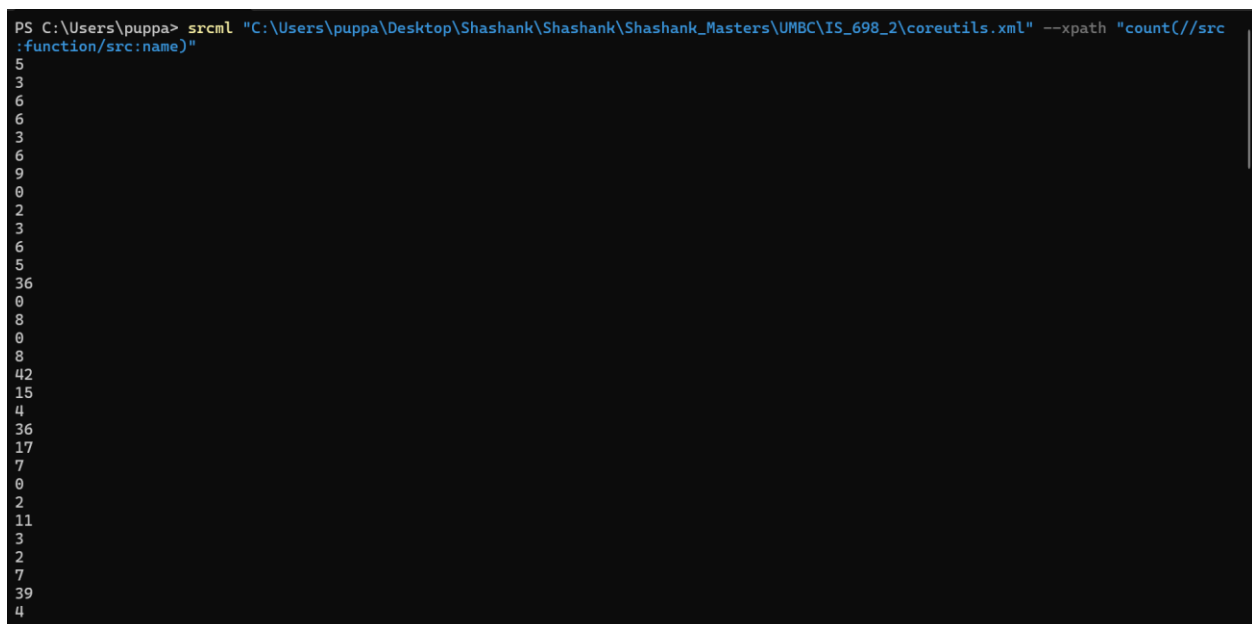
Step 4: Running of the queries to find out the total number of functions in source code and all the functions names

Below is the screenshot showing how the function names are extracted from the coreutils.xml file and stored into the function_names.txt file using the SRCML command.



```
Windows PowerShell
PS C:\Users\puppa> srcml "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils.xml" --xpath "//src:func
tion/src:name/text()" > "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\function_names.txt"
PS C:\Users\puppa> |
```

Next, the query function to calculate or count the number of functions in the function_names.txt file;



```
PS C:\Users\puppa> srcml "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils.xml" --xpath "count(//src
:function/src:name)"
5
3
6
6
3
6
9
0
2
3
6
5
36
0
8
0
8
42
15
4
36
17
7
0
2
11
3
2
7
39
4
```

The count values are all summed up to calculate the count of total number of function names;

```
Windows PowerShell
PS C:\Users\puppa\downloads> (Select-String -Path "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\function_names.txt" -Pattern "item=").Count
1135
PS C:\Users\puppa\downloads>
```

The final count value is then appended into the function_name.txt text file as follows;

```
PS C:\Users\puppa> Add-Content "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\function_names.txt" "Total Number of Functions: $((Select-String -Path 'C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\function_names.txt' -Pattern '<unit').Count)"
PS C:\Users\puppa>
```

Please find below the screenshot of SRCML command and query to find out the Function names from the XML file and create the function_names.txt file;

```
Windows PowerShell
PS C:\Users\puppa> srcml "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils.xml" --xpath "//src:function/src:name/text()" | Select-String -Pattern "item=" | ForEach-Object { $_.Line -replace '.*item="d+">', '' -replace '</unit>', '' } | Set-Content "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\function_names.txt"
PS C:\Users\puppa>
```

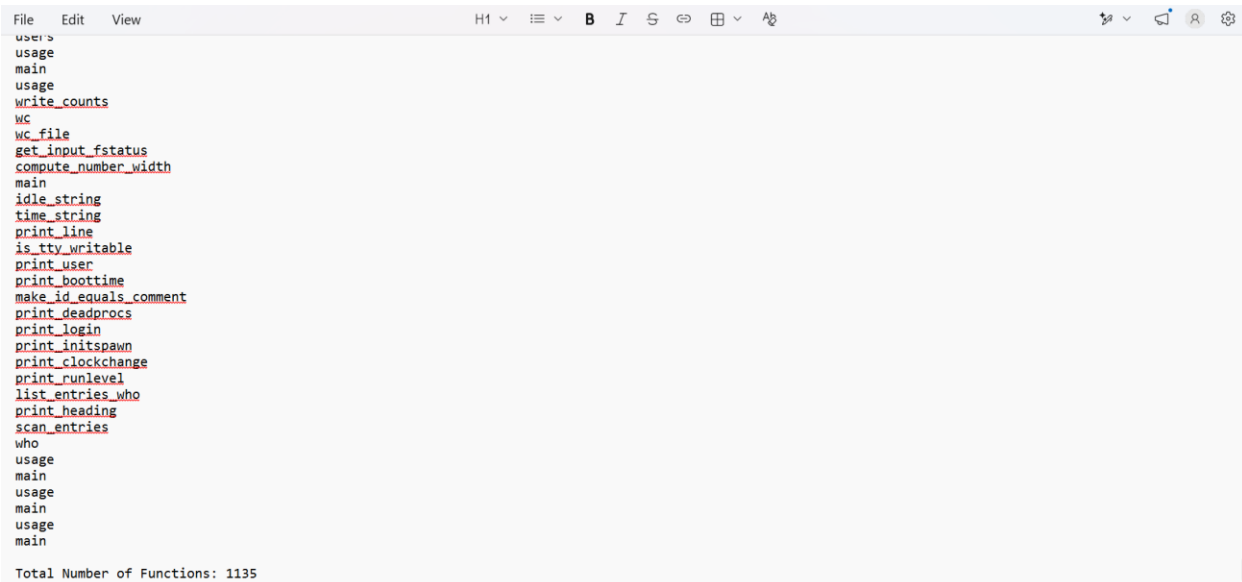
Once the function names are found out, the total number of functions are calculated using the count function;

```
PS C:\Users\puppa> Add-Content "C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\function_names.txt" "`nTotal Number of Functions: $((Get-Content 'C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\function_names.txt' | Measure-Object -Line).Lines)"
PS C:\Users\puppa>
```

The function_names.txt file is as follows;

(The screenshots are continued on the next page...)

```
usage
wrap_write
do_encode
do_decode
main
usage
remove_suffix
main
usage
next_line_num
simple_cat
write_pending
cat
main
compute_context_from_mask
change_file_context
process file
```



```
File Edit View H1 B I S ↺ ↻ ⌵ ⌵
users
usage
main
usage
write_counts
wc
wc_file
get_input_fstatus
compute_number_width
main
idle_string
time_string
print_line
is_tty_writable
print_user
print_boottime
make_id_equals_comment
print_deadprocs
print_login
print_initspawn
print_clockchange
print_runlevel
list_entries_who
print_heading
scan_entries
who
usage
main
usage
main
usage
main

Total Number of Functions: 1135
```

Step 5 (a): The Same Above task with the Generative AI

Now we are going to perform the same tasks using Generative AI, i.e. we are going to feed the actual coreutils source code files to the LLM and perform the operation of extracting the function names and the count, we use **Ollama** which is installed locally to perform the operation using **Llama3.2 3b**

Now to perform the same above task of extracting out the function names from coreutils xml file and count the number using gen AI,

Instructions while running Ollama to extract the function names and count: -

1. Please make sure Python is installed in the system.
2. Ollama is installed
3. Llama3.2 3b model installed (if not run the command of **run: ollama pull llama3.2:3b** to install the model locally)
4. Make sure the coreutils source code files and the python script file are in the same folder
5. Run the Python script using the command – **python HW3_Updated.py**
6. The resultant function names and count are saved as **ollama_function_sample.txt** file in the same folder.

Please find below the screenshot of python code consisting of the prompt used to tell the model what must be done, like listing out and counting the function's names and storing them in a .txt file.

```
HW3_Updated.py X
C:\Users\puppa\Desktop> Shashank > Shashank > Shashank_Masters > UMBC > IS_698_2 > HW3_Updated.py > ...
1 import os
2 import subprocess
3
4 src_path = r"C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils-8.13\src"
5 output_file = r"C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\ollama_functions_sample.txt"
6
7 # Sample of 20 representative .c files
8 sample_files = [
9     'base64.c', 'basename.c', 'cat.c', 'chmod.c', 'chown.c',
10    'cp.c', 'cut.c', 'date.c', 'df.c', 'du.c',
11    'echo.c', 'head.c', 'ls.c', 'mkdir.c', 'mv.c',
12    'pwd.c', 'rm.c', 'sort.c', 'tail.c', 'wc.c'
13 ]
14
15 all_functions = []
16
17 print(f"Processing {len(sample_files)} sample files...")
18 print("="*50)
19
20 for i, c_file in enumerate(sample_files):
21     file_path = os.path.join(src_path, c_file)
22
23     if not os.path.exists(file_path):
24         print(f"Skipping {c_file} - file not found")
25         continue
26
27     with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
28         code = f.read(5000)
29
```

```
HW3_Updated.py X
C:\Users\puppa\Desktop> Shashank > Shashank > Shashank_Masters > UMBC > IS_698_2 > HW3_Updated.py > ...
30 prompt = f"List ONLY the C function names defined in this code, one per line, no explanations, no numbers, no extra text:\n\n{code}"
31
32 print(f"Processing {i+1}/{len(sample_files)}: {c_file}")
33
34 result = subprocess.run(
35     ["ollama", "run", "llama3.2:3b", prompt],
36     capture_output=True,
37     text=True,
38     timeout=300
39 )
40
41 if result.returncode == 0 and result.stdout.strip():
42     functions = [line.strip() for line in result.stdout.strip().split('\n')
43                 if line.strip() and not line.strip().startswith('#')]
44     for func in functions:
45         all_functions.append(f"{c_file}: {func}")
46     print(f" Found {len(functions)} functions")
47 else:
48     print(f" Skipped or error: {result.stderr.strip()}")
49
50 with open(output_file, 'w') as f:
51     f.write("FUNCTION NAMES FOUND BY OLLAMA (llama3.2:3b)\n")
52     f.write("Method: Sample of 20 raw .c files fed directly to Ollama\n")
53     f.write("Note: Full coreutils has 130+ files. Sample used due to local hardware limitations.\n")
54     f.write("="*50 + "\n\n")
55     for func in all_functions:
56         f.write(func + "\n")
57     f.write("\n" + "="*50 + "\n")
58     f.write(f"Total Number of Functions in Sample: {len(all_functions)}\n")
59
```

The Coreutils Src folder has over 139 source code files which cannot be processed because of hardware limitations. As a result, I have used a Sample set of 20 src files and found the function names and count of function names as well. The output is as follows;


```
Windows PowerShell
PS C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2> python HW3_Updated.py
Processing 20 sample files...
=====
Processing 1/20: base64.c
Exception in thread Thread-2 (_readerthread):
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\threading.py", line 1075
    in _bootstrap_inner
    self.run()
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\threading.py", line 1012
    in run
    self._target(*self._args, **self._kwargs)
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\subprocess.py", line 159
    9, in _readerthread
        buffer.append(fh.read())
        ^^^^^^^^^
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\encodings\cp1252.py", li
    ne 23, in decode
        return codecs.charmap_decode(input,self.errors,decoding_table)[0]
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
UnicodeDecodeError: 'charmap' codec can't decode byte 0x8f in position 332: character maps to <undefined>
Found 21 functions
Processing 2/20: basename.c
Exception in thread Thread-4 (_readerthread):
=====
UnicodeDecodeError: 'charmap' codec can't decode byte 0x8f in position 332: character maps to <undefined>
Found 25 functions
Processing 20/20: wc.c
Exception in thread Thread-40 (_readerthread):
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\threading.py", line 1075
    in _bootstrap_inner
    self.run()
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\threading.py", line 1012
    in run
    self._target(*self._args, **self._kwargs)
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\subprocess.py", line 159
    9, in _readerthread
        buffer.append(fh.read())
        ^^^^^^^^^
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12.3.12.2800.0_x64__qbz5n2kfra8p0\Lib\encodings\cp1252.py", li
    ne 23, in decode
        return codecs.charmap_decode(input,self.errors,decoding_table)[0]
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
UnicodeDecodeError: 'charmap' codec can't decode byte 0x8f in position 371: character maps to <undefined>
Found 32 functions
Done! Total functions found: 378
Results saved to: C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\ollama_functions_sample.txt
```

The total number of functions found from the processing of 20 files was 378, and the names and count are stored in the `ollama_functions_sample.txt` file. Please find below the screenshots of the text file;

GitHub Link to the HW3_Updated Code:-

https://github.com/UB01976/Sotware_Engineering_698/blob/main/HW3_Updated.py

(The screenshots are continued on the next page...)


```
FUNCTION NAMES FOUND BY OLLAMA (llama3.2:3b)
Method: Sample of 20 raw .c files fed directly to Ollama
Note: Full coreutils has 130+ files. Sample used due to local hardware limitations.
=====
```

```
base64.c: encode_base64_data
base64.c: decode_base64_data
base64.c: base64_encode
base64.c: base64_decode
base64.c: emit_ancillary_info
base64.c: fopen
base64.c: fclose
base64.c: freopen
base64.c: fseek
base64.c: fputs
base64.c: fprintf
base64.c: getopt_
base64.c: getopt_long_
base64.c: getopt_bool_
base64.c: getopt_string_
base64.c: printf
base64.c: puts
base64.c: printf_f
base64.c: quote
base64.c: xfreopen
base64.c: xstrtol
basename.c: basename
basename.c: usage
basename.c: remove_suffix
basename.c: main
basename.c: base_name
basename.c: strip_trailing_slashes
basename.c: initialize_main
basename.c: set_program_name
```

```
wc.c: quote
wc.c: xfreopen
wc.c: fadvise
wc.c: mbchar
wc.c: error
wc.c: argv_iter
wc.c: system
wc.c: printf
wc.c: fputs
wc.c: fprintf
wc.c: getopt
wc.c: assert
wc.c: iswspace
wc.c: to_uchar
wc.c: isspace
wc.c: max_line_length
wc.c: total_lines
wc.c: total_words
wc.c: total_chars
wc.c: total_bytes
wc.c: print_lines
wc.c: print_words
wc.c: print_chars
wc.c: print_bytes
wc.c: print_linelength
wc.c: number_width
wc.c: have_read_stdin
wc.c: failed
wc.c: st
wc.c: umaxtostr
```

```
=====
Total Number of Functions in Sample: 378
Files Processed: 20
```

So, this analysis can similarly be performed on all the remaining files as well by using the same method of chunking the source code files into smaller sets.

Verification Using the XML File Generated using SRCML: -

To verify if the Ollama can take the XML input file properly or not and perform the evaluation, I have even got the function names and count from the XML file as well using the Llama3.2 3b model. Please find below the screenshots of python script for running the Llama model on XML file;

```
HW3_Python_Ollama.py X
C: > Users > puppa > Desktop > Shashank > Shashank > Shashank_Masters > UMBC > IS_698_2 > HW3_Python_Ollama.py > ...
1 import os
2 import xml.etree.ElementTree as ET
3 import subprocess
4
5 # Automatically uses the directory where the script is located
6 base_dir = os.path.dirname(os.path.abspath(__file__))
7 xml_file = os.path.join(base_dir, "coreutils.xml")
8 output_file = os.path.join(base_dir, "ollama_function_names.txt")
9
10 # Check if XML file exists or is there any error
11 if not os.path.exists(xml_file):
12     print(f"ERROR: coreutils.xml not found in {base_dir}")
13     print("Please place coreutils.xml in the same folder as this script.")
14     exit()
15
16 print(f"Reading XML from: {xml_file}")
17
18 tree = ET.parse(xml_file)
19 root = tree.getroot()
20
21 ns = {'src': 'http://www.srcML.org/srcML/src'}
22
23 functions = root.findall('.//src:function/src:name', ns)
24 function_names = [f.text for f in functions if f.text]
25
26 print(f"Extracted {len(function_names)} function names from XML")
27
28 sample = function_names[:50]
29 prompt = f"Here are some C function names from coreutils. Just confirm they if look like valid C function names and list them:\n\n" + "\n".join(sample)
```

```
HW3_Python_Ollama.py X
C: > Users > puppa > Desktop > Shashank > Shashank > Shashank_Masters > UMBC > IS_698_2 > HW3_Python_Ollama.py > ...
30
31 print("Sending sample to Ollama for verification...")
32
33 result = subprocess.run(
34     ["ollama", "run", "llama3.2:3b", prompt],
35     capture_output=True,
36     text=True,
37     timeout=120
38 )
39
40 print("Ollama Response:")
41 print(result.stdout)
42
43 if result.stderr:
44     print("Errors:")
45     print(result.stderr)
46
47 with open(output_file, 'w') as f:
48     f.write("FUNCTION NAMES EXTRACTED FROM XML AND VERIFIED BY OLLAMA (Llama3.2 3b)\n")
49     f.write("Tool: Ollama (llama3.2:3b)\n")
50     f.write("="*50 + "\n\n")
51     for func in function_names:
52         f.write(func + "\n")
53     f.write("\n" + "="*50 + "\n")
54     f.write(f"Total Number of Functions: {len(function_names)}\n")
55
56 print(f"\nDone! Total functions found: {len(function_names)}")
57 print(f"Results saved to: {output_file}")
```

I have written the python code in such a way that it can be run in any system, just need to have script file and XML file in same directory, and all the required tools like python and ollama to run the code.

Please find below the attached screenshots after running the script;

```
Windows PowerShell
PS C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2> python HW3_Python_Ollama.py
Reading XML from: C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\coreutils.xml
Extracted 1135 function names from XML
Sending sample to Ollama for verification...
```

```
Windows PowerShell
Ollama Response:
Here is the list of function names from coreutils source code:

1. usage
2. wrap_write
3. do_encode
4. do_decode
5. main
6. usage
7. remove_suffix
8. main
9. usage
10. next_line_num
11. simple_cat
12. write_pending
13. cat
14. main
15. compute_context_from_mask
16. change_file_context
17. process_file
18. process_files
19. usage
20. main
21. parse_group
22. usage
23. main
24. mode_changed
25. describe_change
26. process_file
27. process_files
28. usage
29. main
30. chopt_init
```

```
Windows PowerShell
24. mode_changed
25. describe_change
26. process_file
27. process_files
28. usage
29. main
30. chopt_init
31. chopt_free
32. gid_to_name
33. uid_to_name
34. user_group_str
35. describe_change
36. restricted_chown
37. change_file_owner
38. chown_files
39. usage
40. main
41. set_additional_groups
42. usage
43. main
44. fill_r
45. crc_remainder
46. main
47. cksum
48. usage
49. main
50. usage

Done! Total functions found: 1135
Results saved to: C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2\ollama_function_names.txt
PS C:\Users\puppa\Desktop\Shashank\Shashank\Shashank_Masters\UMBC\IS_698_2>
```

Step 5 (b): The Same Above task with the ChatGpt and Claude

Similar to the above two steps, now using the ChatGPT and Claude for finding out the function names and count. But before doing that, we must design the prompt that has to be given the AI tools which is as follows;

Prompt Design: -

The prompt has been designed in such a way that the AI model assumes the role of the analyst and performs the structured analysis on the code. The model is exclusively asked on how the output should be and what are the constraints to be considered. It is even mentioned to get the functions names and their count while minimizing most of its ambiguity.

Prompt: -

Consider yourself as a software analysis assistant. You need to perform the following Task: Analyze the following C source code and extract all function names defined in this file. Follow the following Instructions:

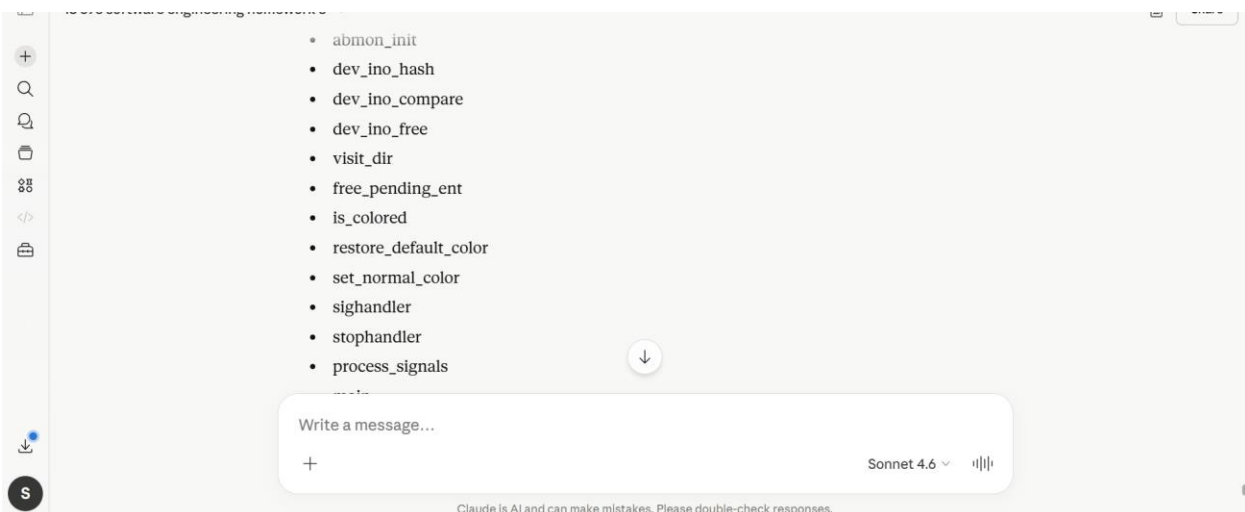
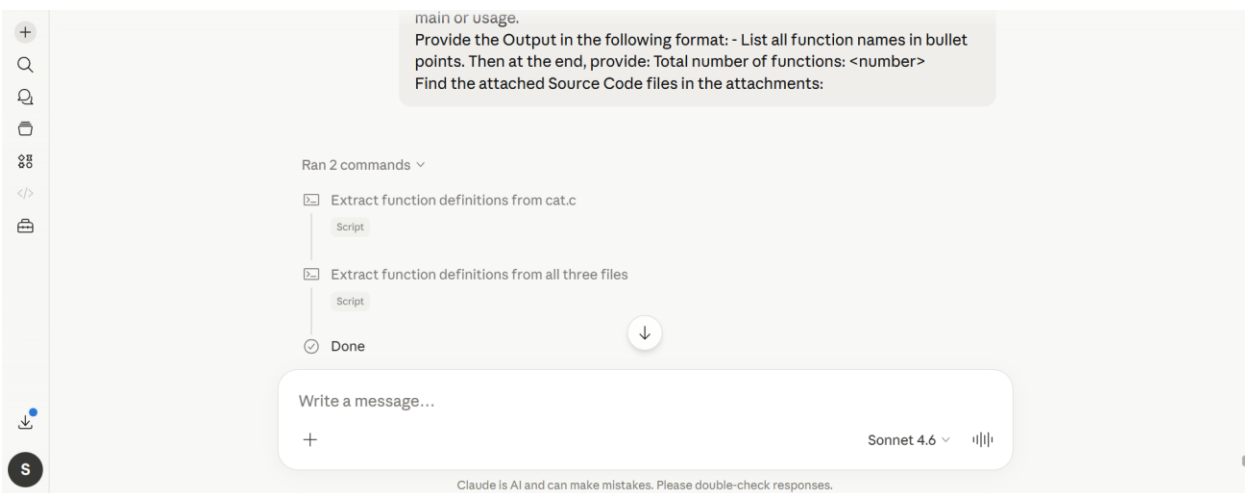
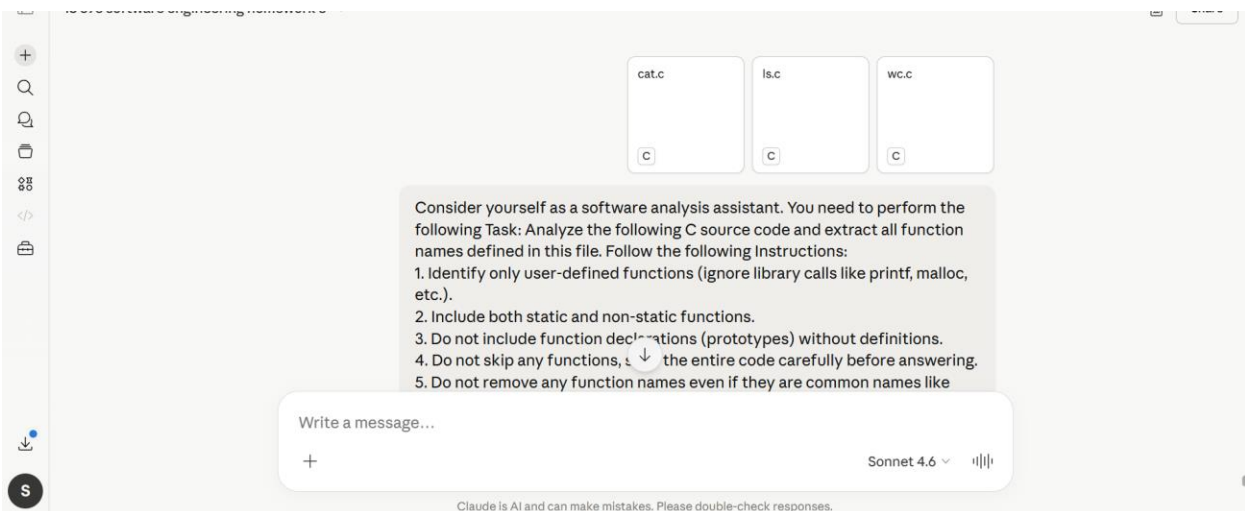
1. Identify only user-defined functions (ignore library calls like printf, malloc, etc.).
2. Include both static and non-static functions.
3. Do not include function declarations (prototypes) without definitions.
4. Do not skip any functions, scan the entire code carefully before answering.
5. Do not remove any function names even if they are common names like main or usage.

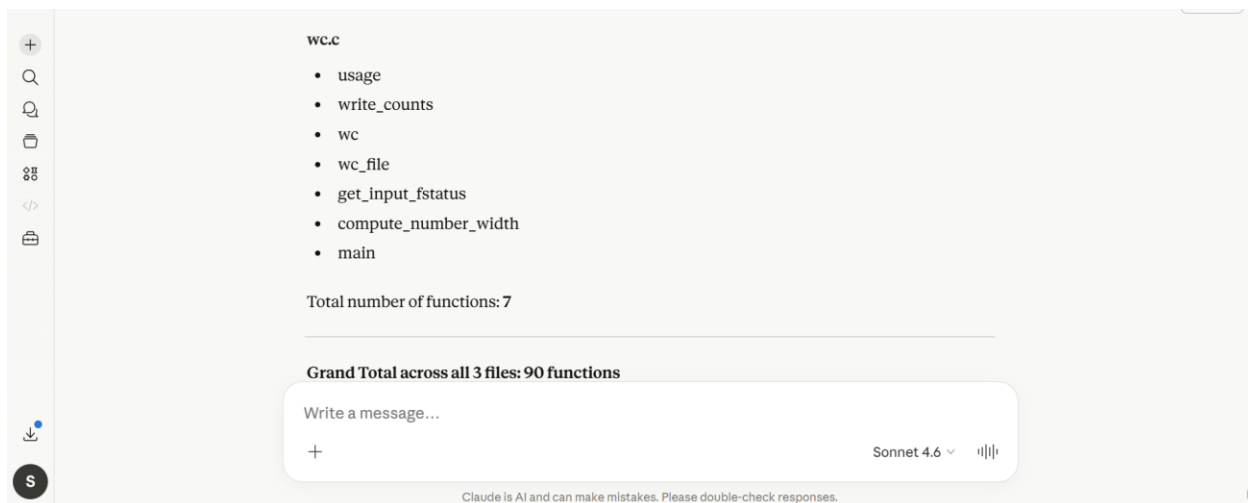
Provide the Output in the following format: - List all function names in bullet points. Then at the end, provide: Total number of functions: <number>

Find the attached Source Code files in the attachments:

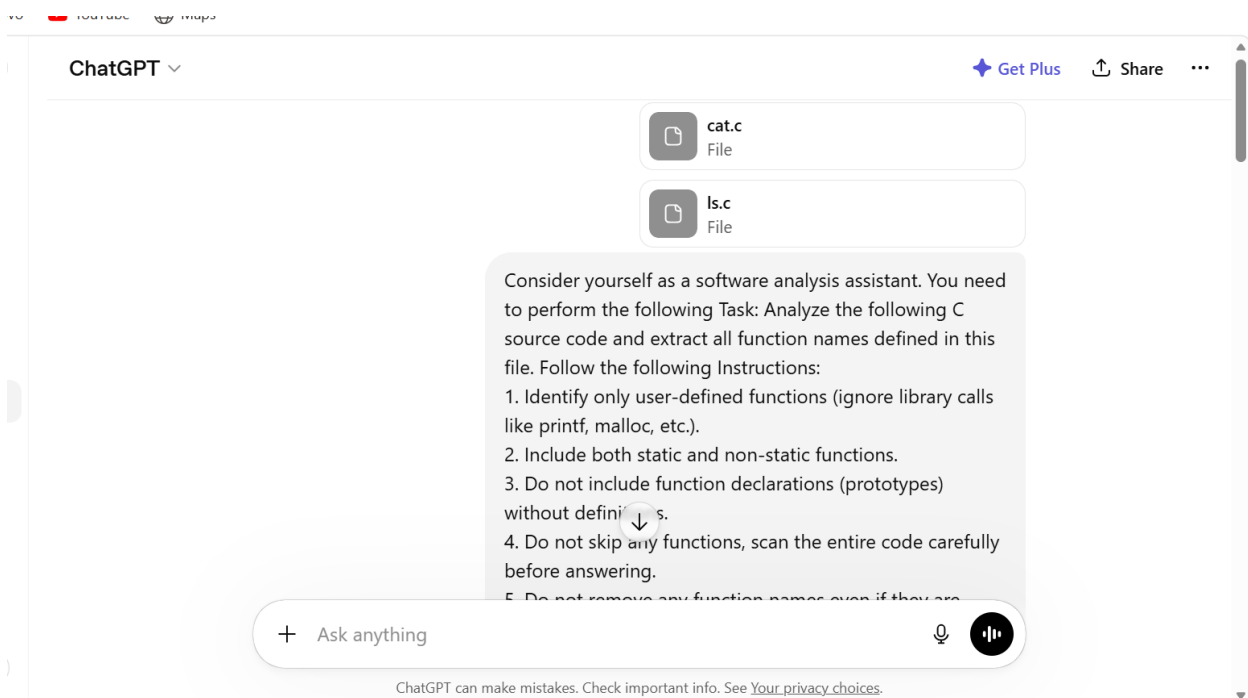
Output: - First the Prompt is tested with Claude and please find the attached screenshot below of the prompt provided, and the corresponding response from Claude (Because of Limitations in uploading the files to Claude, only 3 files are attached at a time for processing, i.e. only a sample of src files are provided to Claude not the complete 130 source code files)

(Please find the attached screenshots on the next page...)





Output: - For ChatGpt, same prompt is provided, and the following output has been observed;



(Note: - As I had ChatGPT's Free version only, I was able to upload only 2 src files and count the functions only from those 2 files)



Functions found in `cat.c`

- `usage`
- `next_line_num`
- `simple_cat`
- `write_pending`
- `cat`
- `main`

Total number of functions in `cat.c` : 6

Functions found in `ls.c`



- `dev_ino_pop`
- `dired_dump_obstack`

 Ask anything



ChatGPT can make mistakes. Check important info. See [Your privacy choices](#).

ChatGPT

[Get Plus](#)  [Share](#)

- `length_of_file_name_and_trails`
- `print_many_per_line`
- `print_horizontal`
- `print_with_commas`
- `indent`
- `attach`
- `init_column_info`
- `calculate_columns`
- `usage`

Total number of functions in `ls.c` : 72

Overall Total



Total number of functions: 78

 Ask anything



ChatGPT can make mistakes. Check important info. See [Your privacy choices](#).

Step 6: Comparative Analysis

All the 3 results are compared on the following evaluation metrics as follows;

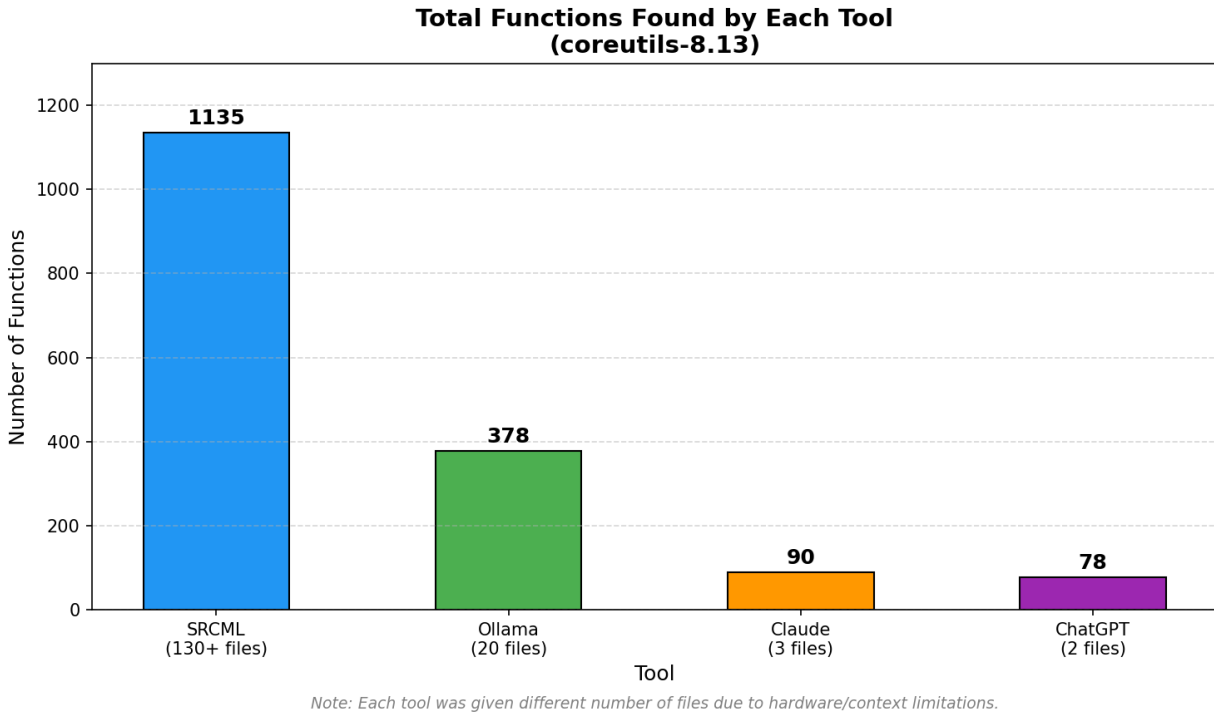
Evaluation Metrics - (1) Accuracy – SRCML is a dedicated parser tool, so it will be able to get the function names and count accurately as it understands the src files C syntax and will not guess any of the function names. Whereas when it comes to genAI (I have used Llama3.23 3b) it is taking really long enough to get the function names from the src files, and that too it is hallucinating. Despite mentioning in the prompt clearly to avoid the function names not in the file, it started to still include the function names which were nowhere present in the src files. Claude and ChatGPT were better in terms of accuracy, because though they were provided with limited number of files of only 2 or 3 src files but they still were able to find out the function names and count accurately. So, in terms of **Accuracy it is SRCML** which was better of all, followed by Claude, ChatGPT, and GenAI.

(2) Recall (Completeness) - SRCML was able to find out all the function names from the XML file. But when it came to GenAI tool, it could not process the complete src files to find the function names. Because of lack of proper hardware requirements to process the 130+ files using ollama, I have given it a sample of 20 src files and made it process. ChatGPT and Claude were able to process completely the given files to it, but again because of the limitation in attaching the files even though it was not provided with all the src files. Hence, even in terms of **Recall, it is SRCML** which is best among all, followed by Claude, ChatGPT, and GenAI.

(3) Ease of Access – GenAI (Ollama) requires the knowledge of installing it and running using Python, which is quite difficult for someone who doesn't understand coding. Even SRCML installation and using the query commands requires knowledge, whereas ChatGPT or Claude either of them requires prompt, which is comparatively easy. So, in terms of **Ease, its ChatGPT and Claude**, followed by SRCML and then GenAI.

(4) Efficiency – Out of all the systems, SRCML was able to perform the operation within a very less amount of time, whereas Llama3.2 3b (GenAI) took around 4 to 5 minutes to just process 1 src file. Compared to GenAI, Claude was better in efficiency and then followed by ChatGPT. So, in terms of **Efficiency it is again SRCML** followed by Claude, ChatGPT, and then GenAI.

Differences in the Results: - As there were limited number of files being executed by Llama3.2 3b and ChatGPT or Claude, so this is how the graph looks like for the total number of functions;

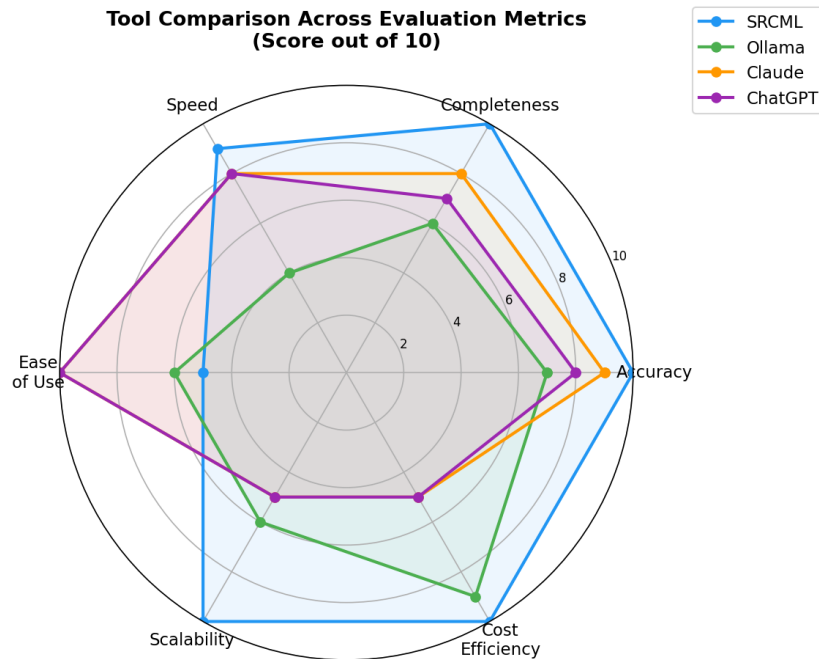


Comparison between all the tools in terms of performance is as follows;

Criteria	SRCML	Ollama	Claude	ChatGPT
Total Functions Found	1135	378	90	78
Files Processed	139	20	3	2
cat.c Functions	6	N/A	6	6
ls.c Functions	77	N/A	77	72
wc.c Functions	7	N/A	7	N/A
Accuracy vs SRCML	100%	~78%	95.67%	93.97%

It can be clearly observed that SRCML being better tool in terms of static analysis than in comparison with others. As it can easily understand the .c files and do the parsing accordingly, it's easy for it.

Overall Evaluation and Personal Insights – On the whole SRCML results can be considered as ground truth because it gave the exact number of function names from the src files as it understands the schematics of .c files. But the numbers or results gave me a different picture, as I expected genAI model like Llama3.2 to easily perform these tasks, but it became quite difficult for it to even process the source code files and get the function names from it. Below is the radar graph comparing all the tools on the score of 10 for all the metrics.



SRCML did not guess any of the function names; it parsed the code similar to the way a compiler does. That's the reason it got accurate results. But when it came to GenAI it had to understand the code and go line by line in the provided src file and had to pull out the function name. While doing this, there is always a risk of hallucination and errors, because LLM models sometimes assume the names instead of truly retrieving them.

Claude performed comparatively better than ChatGPT, as it took time to extract the function names and count them by analyzing the complete prompt and files provided. ChatGPT was not so accurate as it did not take the prompt properly and started to miss the functions.

Conclusion – This is not like competition between all the tools, as every tool has its own significance. But when it comes to **static analysis clearly, SRCML outperformed the GenAI and LLM tools** like ChatGPT and Claude. But when it comes to analysis and interpretation clearly, AI tools are better than compared to SRCML, which does not provide any of those. Therefore, a better approach among all would be to use both the tools in a hybrid approach, i.e. SRCML for static analysis and GenAI for interpretation which would balance the work.